

SE1020 – Object Oriented Programming

Assignment: Project | Group: G20

**Fitness Center
membership Management
System**

Contents

1. Introduction	3
2. Project Overview	4
3. Workload distribution	6
4. Technology used	8
5. System architecture.....	9
6. Class diagram	12
7. OOP Concepts Implementation	13
Encapsulation	13
Inheritance	14
Polymorphism	14
Abstraction	14
8. CRUD Implementation.....	16
9. System Functionality	19
10. User interface	20
11. File Handling.....	23
12. Testing and Validation	25
13. GitHub Repository	27
14. Conclusion.....	28

1. Introduction

The Fitness Center Membership Management System is a web-based application developed using Java and Spring Boot technologies to help fitness centers manage their daily operations efficiently. The system enables administrators to manage member registrations, payment records, workout and diet plans, and membership renewal requests through an organized and user-friendly platform. Members can register, view assigned plans, and maintain their membership details, while administrators can monitor and control overall system activities. The system also incorporates data structures and algorithms, such as queue-based processing and sorting techniques, to enhance efficiency and performance.

The system is necessary because traditional fitness center management methods are often manual, time-consuming, and difficult to maintain. Managing member records, payment tracking, and renewal processing manually can lead to data loss, delays, and operational inefficiencies. This system digitizes these processes to simplify management tasks, reduce errors, and improve the overall efficiency of fitness center operations.

Practical applications of this system include its use by gym administrators, fitness trainers, and fitness center staff to organize member information, manage payments, assign workout and diet plans, and process membership renewals efficiently. The renewal process is further optimized using a queue-based (FIFO) system, while member prioritization is improved through the use of the Insertion Sort algorithm based on membership expiry dates. The system also allows fitness centers to maintain accurate records and provide better service management through a centralized digital platform.

2. Project Overview

The Fitness Center Membership Management System is a modular web application in which different modules are designed to handle specific functionalities of the system. The system is composed of several interconnected modules, ensuring efficient and organized management of fitness center operations.

The **Member Management module** enables administrators to register, update, view, and delete member details while maintaining accurate member records. This module also supports member login and profile management functionalities.

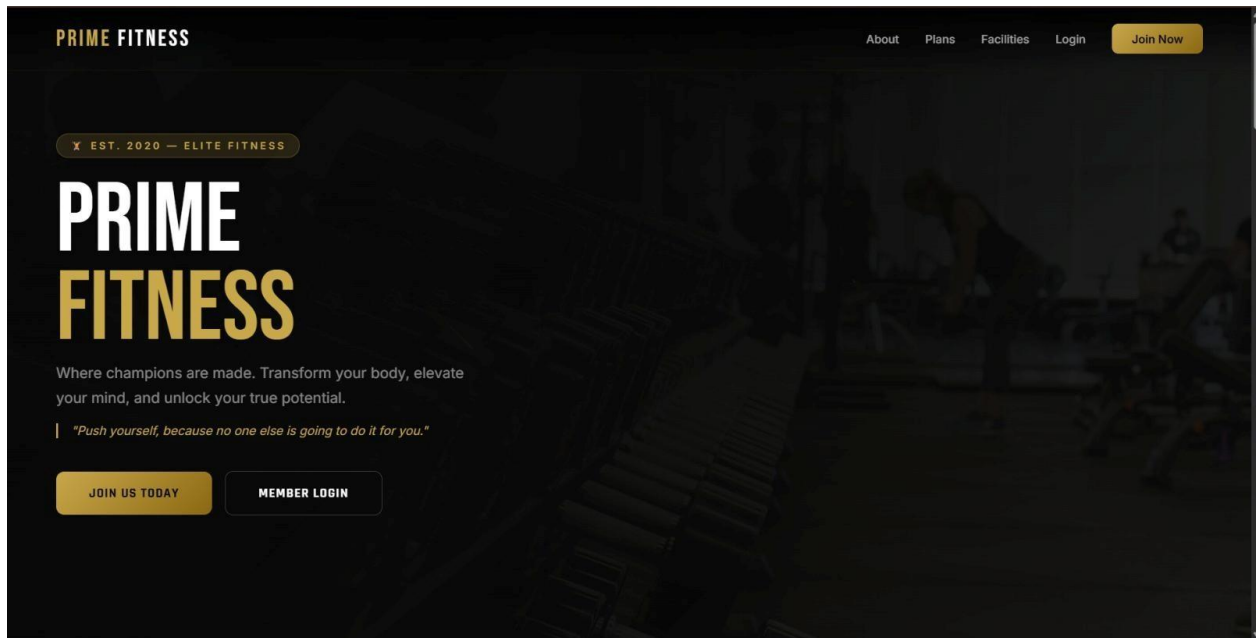
The **Admin Management module** provides system administrators with the ability to manage admin accounts, control system access, and oversee overall system activities. It ensures secure authentication and proper management of user roles.

The **Payment Management module** handles the recording, updating, and tracking of membership payments, ensuring accurate financial management. It allows administrators to monitor payment status and generate revenue-related insights.

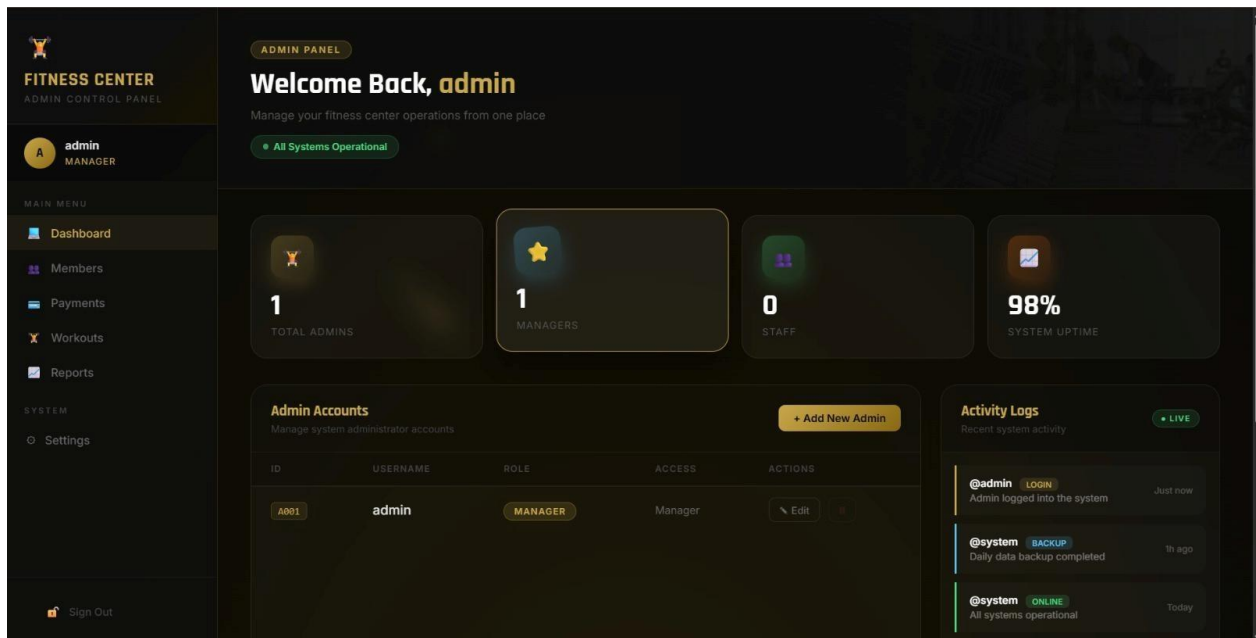
The system also includes the **Workout and Diet Plan Management module**, which enables administrators to assign, update, and manage personalized workout routines and diet plans for members. The workout system includes predefined templates such as Push-Pull-Legs and Full Body routines, which can be customized based on member requirements.

In addition, the **Membership Renewal Management module** is responsible for handling renewal requests using a queue-based (FIFO) data structure. This ensures that renewal requests are processed fairly in the order they are received. Furthermore, the system uses the **Insertion Sort algorithm** to prioritize members based on their membership expiry dates, improving the efficiency of renewal management.

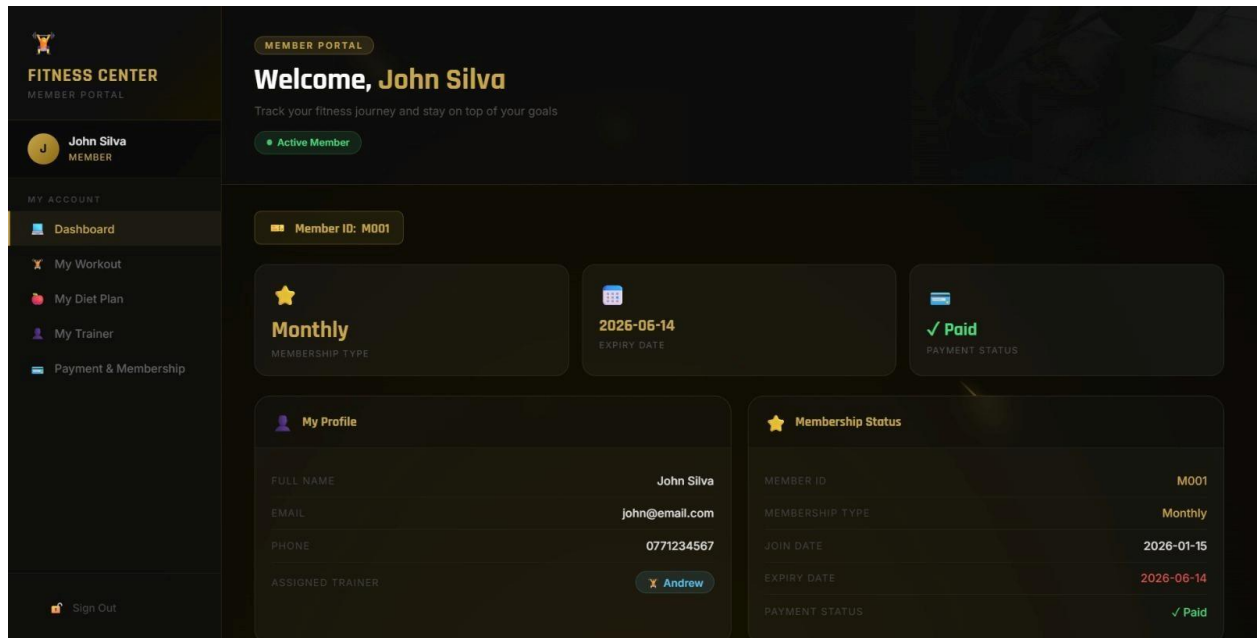
All modules are implemented using CRUD (Create, Read, Update, Delete) operations and developed following Object-Oriented Programming (OOP) principles and the Model-View-Controller (MVC) architecture. This enhances modularity, scalability, maintainability, and overall system organization.



(figure 1- Home page)



(figure 2- Admin dashboard)



(figure3- Member dashboard)

3. Workload distribution

This system was designed as a group project, where the overall functionality was divided into several modules. Each member of the group was assigned a specific module and was responsible for developing its functionality using CRUD (Create, Read, Update, Delete) operations based on Object-Oriented Programming (OOP) principles and the Model-View-Controller (MVC) architecture.

This approach ensured equal contribution from all team members and supported the systematic and structured development of the project.

The modules were developed individually by each team member and later integrated to form the complete Fitness Center Membership Management System. This modular development approach improved maintainability, reduced development complexity, and allowed efficient collaboration among team members.

The **renewal module**, developed as part of the system, incorporates a **queue (FIFO) data structure** to process membership renewal requests in a fair and organized order. In addition, the **Insertion Sort algorithm** is used to prioritize members based on their membership expiry dates, improving the efficiency of renewal handling.

The **workout module** includes predefined workout templates such as Push-Pull-Legs and Full Body routines, allowing structured and customizable plan management. Similarly, the diet plan module enables personalized nutrition planning for members.

Overall, this distribution of responsibilities ensured that each module was developed efficiently while maintaining consistency and integration across the entire system.

Student name	Module	Responsibilities	CRUD Operations
IT25101736 Sooriyaarachchi A.G.S.T	Member Management	Manage member details including registration, profile information, fitness data, and membership records.	Create → Add new member Read → View/search member details Update → Modify member information Delete → Remove member account
IT25101719 Jayawardena S.D.D.L	Admin Management	Manage administrator accounts and overall system control functions.	Create → Add admin, member accounts Read → View admin, member details Update → Modify admin, member profiles Delete → Remove admin, member accounts

IT25101711 Madugalle E.W.M.W.S.A.K	Workout & Diet Management	Manage workout schedules and diet plans assigned to members. Includes predefined workout templates and structured plan management	Create → Assign workout/diet plans Read → View assigned plans Update → Modify plan details Delete → Remove outdated plans
IT25101763 Wijewardana D.S	Payment Management	Manage membership payment records and payment status tracking and revenue monitoring.	Create → Add payment records Read → View payment history Update → Update payment status/details Delete → Remove invalid payment records
IT25101744 Wijewardhana A.A.G.S.U	Membership Renewal	Handle membership renewal requests using a queue-based (FIFO) system and prioritize members using an insertion sort algorithm	Create → Add renewal requests Read → View renewal queue Update → Process renewal requests Delete → Remove completed requests

4. Technology used

The Fitness Center Membership Management System has been developed using modern web technologies to provide efficient processing and ease of use. The system uses **Java Spring Boot** for backend development, which supports Object-Oriented Programming (OOP) principles such as encapsulation, inheritance, polymorphism, and abstraction.

The application follows the **Model-View-Controller (MVC) architecture**, where Spring Boot controllers are used to handle HTTP requests and responses. These controllers interact with the service layer, which implements the business logic and manages system operations.

The user interface of the application is developed using **HTML, CSS, and JavaScript**, while **Bootstrap** is used to ensure responsiveness and enhance the visual appearance of the system across different devices.

For data storage and retrieval, the system uses **file handling** techniques, storing and reading data from structured text files (.txt). Java file input/output streams such as **BufferedReader** and **BufferedWriter** are used to efficiently manage data operations. This approach eliminates the need for a database, as required by the assignment.

The Integrated Development Environment (IDE) used for this project is **IntelliJ IDEA**, which provides powerful tools for Spring Boot application development, debugging, and project management.

5. System architecture

The Fitness Center Membership Management System is designed using a layered architecture, which follows the Model-View-Controller (MVC) pattern. The system consists of the presentation layer, controller layer, service layer, and data layer, ensuring a clear separation of concerns.

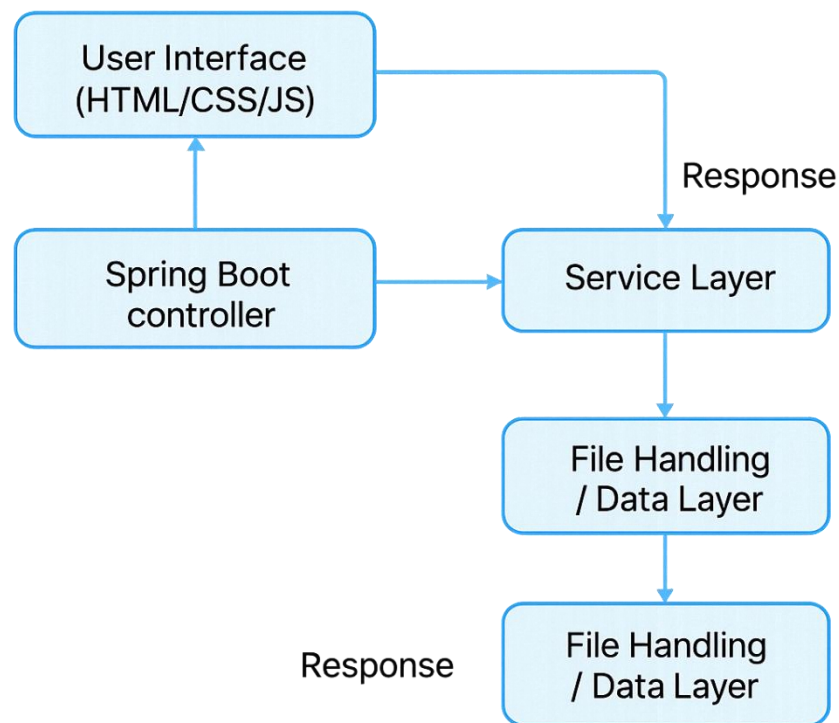
The system is accessed through the frontend interface developed using HTML, CSS, and JavaScript. When a user interacts with the system through form submissions or data requests, the request is handled by the Spring Boot controllers, which act as request handlers responsible for processing client inputs.

The controllers communicate with the service layer, where the business logic is implemented. This layer handles operations such as creating, reading, updating, and deleting (CRUD) system data. Additional logic such as workout plan assignment, diet plan management, renewal request processing, and payment tracking is also handled within the service layer.

The renewal management module incorporates a queue-based (FIFO) data structure to process renewal requests fairly, while the insertion sort algorithm is used to organize members based on their membership expiry dates.

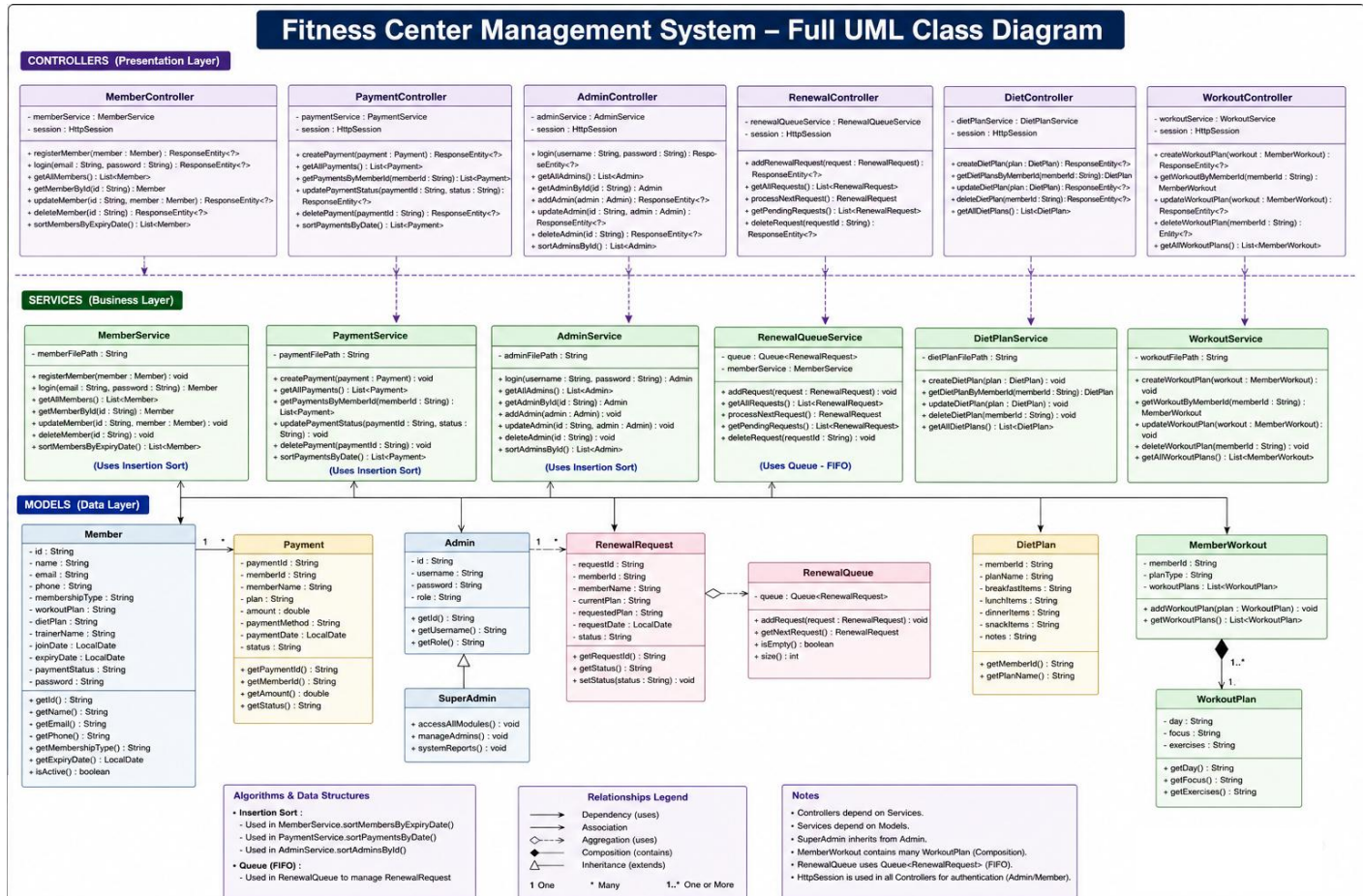
These operations are carried out through file handling, where data is stored and retrieved from structured text files. Each module maintains its own file for efficient data organization.

After processing the request, the response is returned to the frontend interface and displayed to the user. This layered architecture improves system modularity, maintainability, scalability, and overall efficiency.



(Figure 4- Simple architecture diagram)

6. Class diagram



(Figure 5- Class diagram)

The class diagram represents the overall design of the Fitness Center Management System based on the Model-View-Controller (MVC) architecture. It illustrates the relationships between controllers, services, and model classes, clearly separating the presentation layer, business logic layer, and data layer.

The system is structured into multiple controllers, including MemberController, PaymentController, AdminController, RenewalController, DietController, and WorkoutController. These controller classes handle user requests and interact with their respective service classes to process system operations.

The service layer contains classes such as `MemberService`, `PaymentService`, `AdminService`, `RenewalQueueService`, `DietPlanService`, and `WorkoutService`. These services implement the core business logic of the system, including member management, payment processing, diet and workout plan handling, and renewal request management. The services also manage file-based data operations for storing and retrieving information.

The model layer represents the data structure of the system, including classes such as `Member`, `Payment`, `Admin`, `SuperAdmin`, `RenewalRequest`, `RenewalQueue`, `DietPlan`, `MemberWorkout`, and `WorkoutPlan`. These classes store the system data and provide methods to access and modify it.

Inheritance is demonstrated between the `Admin` and `SuperAdmin` classes, where `SuperAdmin` extends the `Admin` class and provides extended administrative functionality.

Composition relationships are used in the workout module, where a `MemberWorkout` object contains a list of `WorkoutPlan` objects, representing multiple workout sessions assigned to a member.

The renewal management system utilizes a queue-based (FIFO) data structure through the `RenewalQueue` class, ensuring that renewal requests are processed in the order they are received. Additionally, the `InsertionSort` algorithm is used within the service layer to sort members based on their membership expiry dates, allowing efficient prioritization of renewals.

Overall, the class diagram illustrates how the system components interact with each other in a structured and modular way, ensuring scalability, maintainability, and efficient management of fitness center operations.

7. OOP Concepts Implementation

The system incorporates key Object-Oriented Programming (OOP) principles to ensure a wellstructured, modular, and maintainable design.

Encapsulation

Encapsulation is implemented by defining private attributes within model classes such as **`Member`**, **`Admin`**, **`Payment`**, **`DietPlan`**, **`RenewalRequest`**, and **`WorkoutPlan`**. Public getter and setter methods are used to access and update these attributes in a controlled manner.

This approach protects data integrity by restricting direct access to class variables and ensures that modifications are performed safely through defined methods.

Inheritance

Inheritance is implemented in the system through the relationship between the **Admin** and **SuperAdmin** classes, where the SuperAdmin class extends the Admin class. This allows the SuperAdmin to inherit common properties such as ID, username, password, and role, while also introducing additional functionalities such as full system access and enhanced administrative control.

Polymorphism

Polymorphism is achieved through method overriding in the Admin hierarchy. For example, the **getAccessLevel()** method is implemented in both Admin and SuperAdmin classes, where the SuperAdmin class provides a different implementation to reflect higher-level privileges. This allows the system to dynamically determine the behavior of objects at runtime based on their specific type.

Abstraction

Abstraction is implemented through the use of service classes such as **MemberService**, **PaymentService**, **AdminService**, **RenewalQueueService**, **DietPlanService**, and **WorkoutService**. These classes encapsulate the business logic of the system and provide simplified methods for controllers to perform operations without exposing internal implementation details.

This separation ensures that controllers interact only with necessary functionalities, improving code maintainability and readability.

```
public class member {  
  
    private String memberId;  
    private String name;  
    private String email;  
    private String password;  
    private int Weight;  
    private int height;  
    private String role; //user or admin
```

(Figure 6- Java Class with private variables member.java)

```
    public String getMemberId() {  
        return memberId;  
    }  
  
    public void setMemberId(String memberId) {  
        this.memberId = memberId;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
}
```

(Figure 7- Getter & Setter methods member.java)

8. CRUD Implementation

CRUD (Create, Read, Update, Delete) operations are implemented throughout all modules of the system to efficiently manage data. These operations are performed using Spring Boot controllers along with file handling techniques to store, retrieve, update, and delete data in text files.

Within the **Member Management module**, new members can be registered by entering details such as name, contact information, and membership details. The system allows viewing and searching member records, updating member information, and deleting member profiles when necessary.

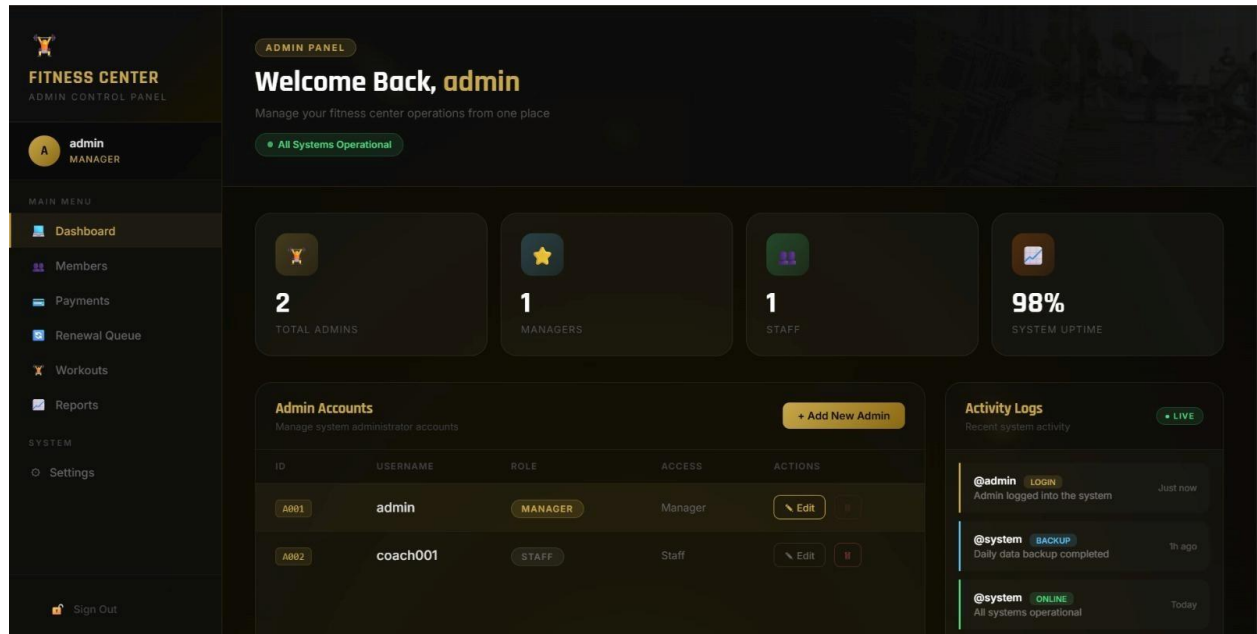
In the **Workout and Diet Plan Management module**, administrators can create and assign workout and diet plans for members. Workout plans include predefined templates such as Push-Pull-Legs and Full Body routines, which can be customized. These plans can be viewed, updated, and removed as required.

In the **Payment Management module**, payment records are created when members make payments for gym memberships or subscriptions. Users can view payment details, update payment statuses, and delete incorrect or outdated payment records.

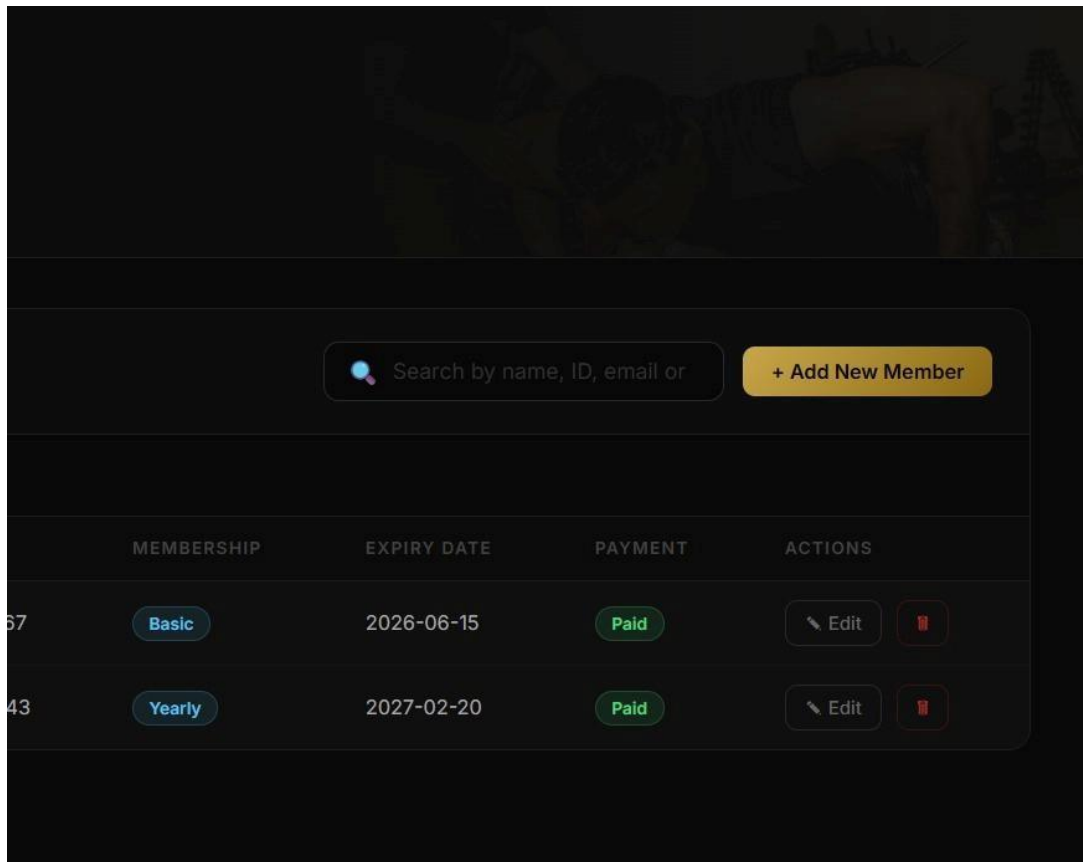
The **Renewal Management module** allows members to submit renewal requests, which are stored and processed using a queue-based (FIFO) approach. Administrators can view pending requests, process them in order, and update their status accordingly.

The **Admin Management module** enables administrators to manage system records, including creating, updating, and deleting admin accounts, as well as monitoring system activities.

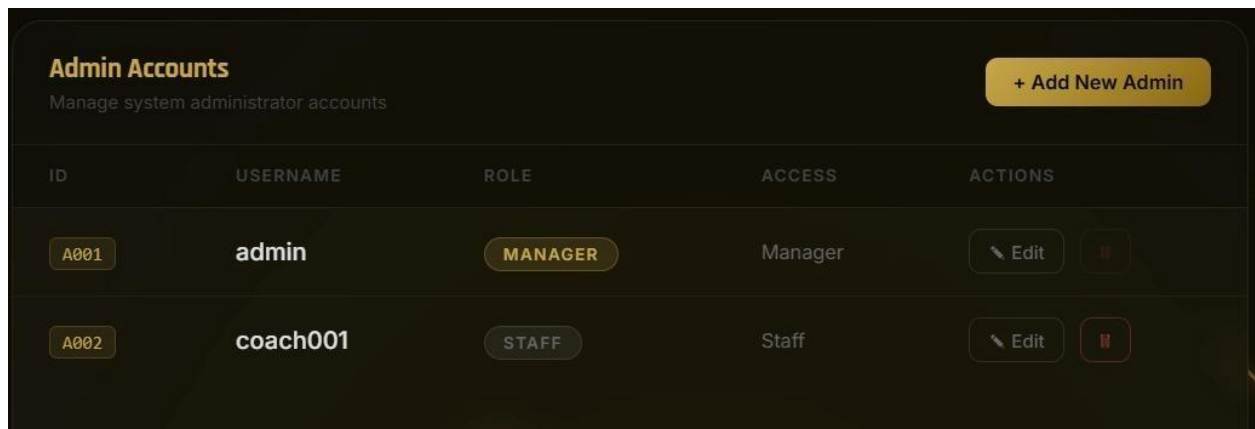
All CRUD operations are implemented through the service layer, which handles business logic and interacts with file storage. This ensures flexible, efficient, and dynamic data management across all system functionalities, allowing smooth operation and easy maintenance of the fitness center management system.



(Figure 8- Admin)



(Figure 9- Add, Edit, delete member details)



(Figure 10- Add, Edit, Delete Admin details)

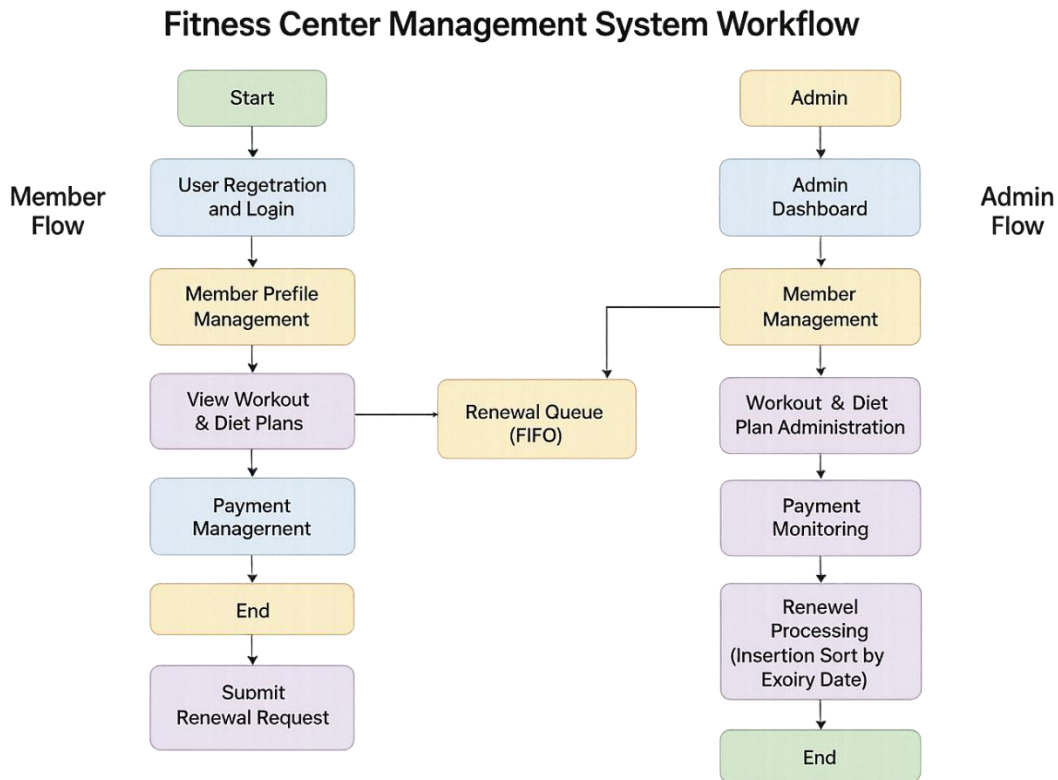
9. System Functionality

The system provides a complete and efficient process for fitness center management. Firstly, members register and log into the system using their credentials. Once logged in, members can access their profiles, view their assigned workout and diet plans, and manage their membership details. Members can also make payments and update their personal information when required.

The system also provides an administrative panel where administrators can manage all system data, including members, workout and diet plans, payment records, and renewal requests. This ensures smooth operation, proper monitoring, and effective management of the entire fitness center.

The system workflow can be broken down into the following processes:

- **User Registration and Login** – Members create accounts and securely log into the system.
- **Member Profile Management** – Members can view, update, and manage their personal and membership details.
- **Workout Plan Management** – Members can view assigned workout plans, while administrators can assign and customize workout plans using predefined templates such as Push-Pull-Legs and Full Body routines.
- **Diet Plan Management** – Members can manage their diet plans, and administrators can create, update, and monitor nutrition plans for all members.
- **Payment Management** – Members can make payments, view payment history, and manage subscription details, while administrators track payment status and revenue.
- **Renewal Request Processing** – Members can submit renewal requests, which are processed using a queue-based (FIFO) system to ensure fair handling.
- **Member Prioritization** – The system uses the Insertion Sort algorithm to organize members based on their membership expiry dates for efficient renewal management.
- **Admin Management** – Administrators can manage admin accounts, member records, and oversee all system activities.



(Figure 11- Work flow diagram)

10. User interface

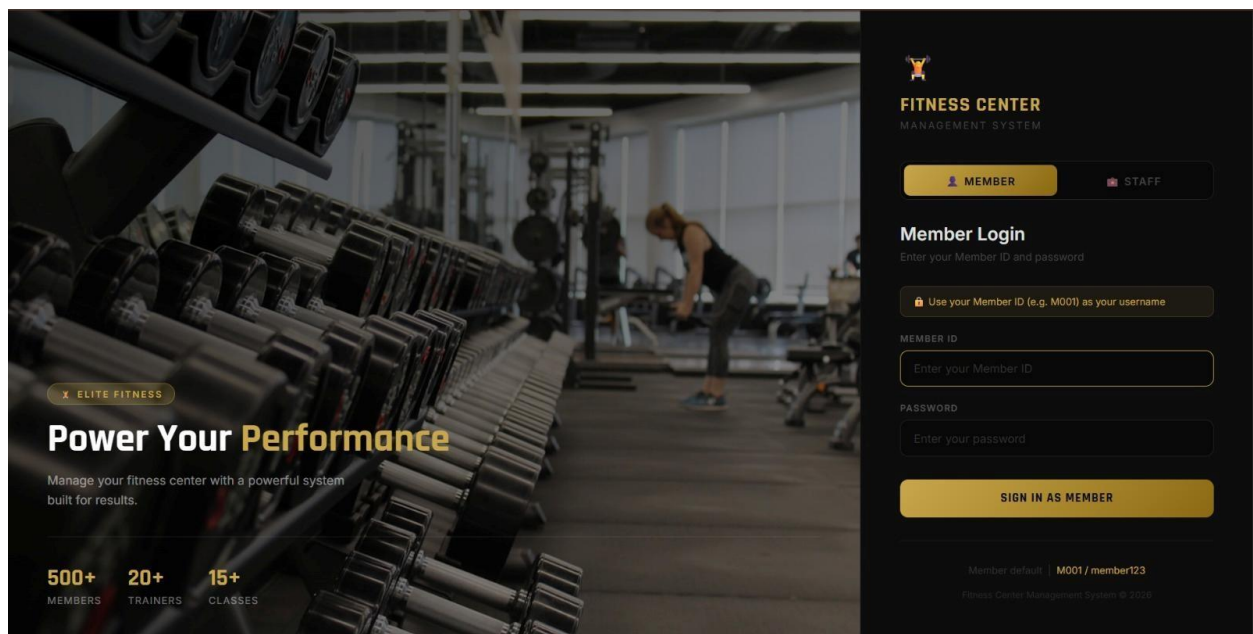
The system features an intuitive and user-friendly interface designed for both members and administrators. The user interface is developed using HTML and CSS for structure and styling, while Bootstrap is used to ensure responsiveness and enhance the overall visual appearance of the system.

The interface is designed to support smooth navigation between different modules, including member management, workout and diet plan management, payment processing, and renewal request handling. Users can easily access these features through structured menus and clearly organized pages.

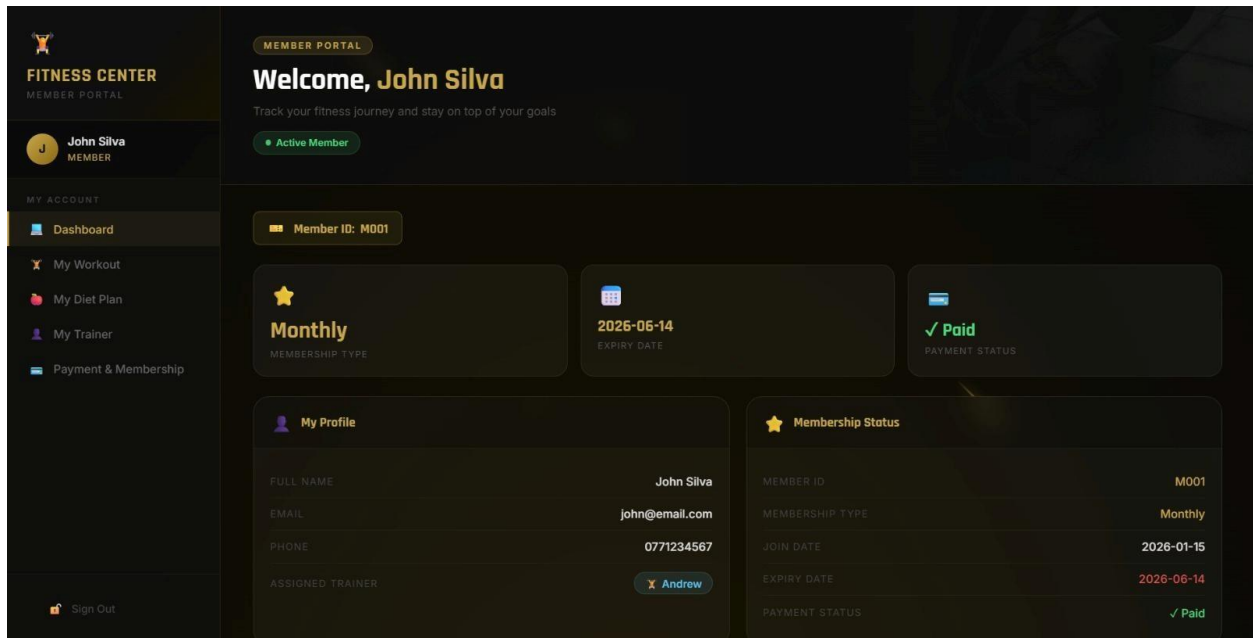
Separate interfaces are provided for members and administrators. Members can register, log in, view and update their profiles, access workout and diet plans, make payments, and submit renewal requests. Administrators are provided with a dashboard that allows them to manage members, assign workout and diet plans, monitor payments, and process renewal requests efficiently.

The system includes well-designed forms for performing CRUD operations such as registering members, updating profiles, assigning plans, and recording payments. Tables are used to display data such as member lists, payment records, and renewal requests in a clear and organized manner. The user interface is fully responsive, allowing users to access the system efficiently through both desktop and mobile devices. Bootstrap components ensure adaptability across different screen sizes and enhance usability.

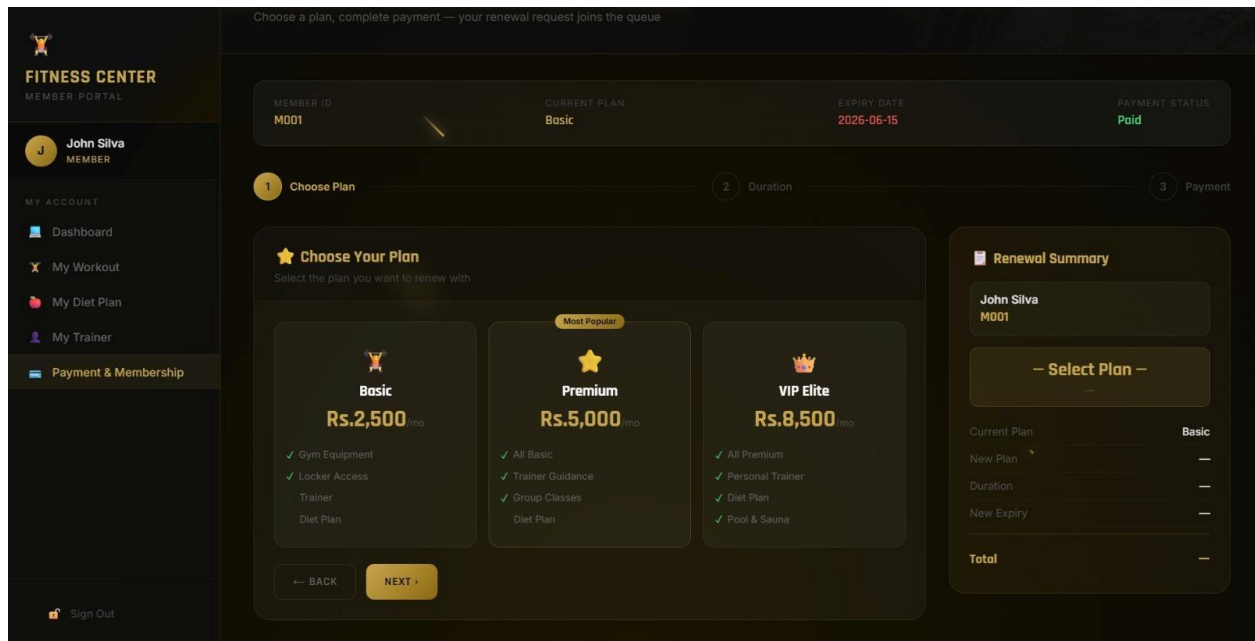
Special attention has been given to the design of forms, buttons, tables, and layouts to maintain a clean, professional, and consistent appearance. This ensures a seamless user experience and makes the system easy to use for managing fitness center activities.



(Figure 12 -Login page)



(Figure 13- Customer Dashboard)



(Figure 14- Membership Plans)

Renew Your Membership
Choose a plan, complete payment — your renewal request joins the queue

MEMBER ID: M001 | CURRENT PLAN: Basic | EXPIRY DATE: 2026-06-15 | PAYMENT STATUS: Paid

1 Choose Plan | 2 Duration | 3 Payment

Payment Method
Choose how you would like to pay

- Card (Credit / Debit)
- Bank Transfer (Direct deposit)
- Cash (Pay at counter)

Renewal Summary

John Silva
M001

Basic
MONTHLY PLAN

Current Plan	Basic
New Plan	Basic
Duration	Monthly
New Expiry	2026-06-16
Total	Rs.2,500

(Figure 15-Payments)

11. File Handling

```

1 |M001|John Silva|john@email.com|0771234567|Basic|Full Body Workout|Keto Diet|Andrew|2026-01-15|2026-06-15|Paid|member123
2 |M002|Sarah Fernando|sarah@email.com|0779876543|Yearly|Cardio Plan|Low Carb Diet|Michael|2026-02-20|2027-02-20|Paid|member456
3

```

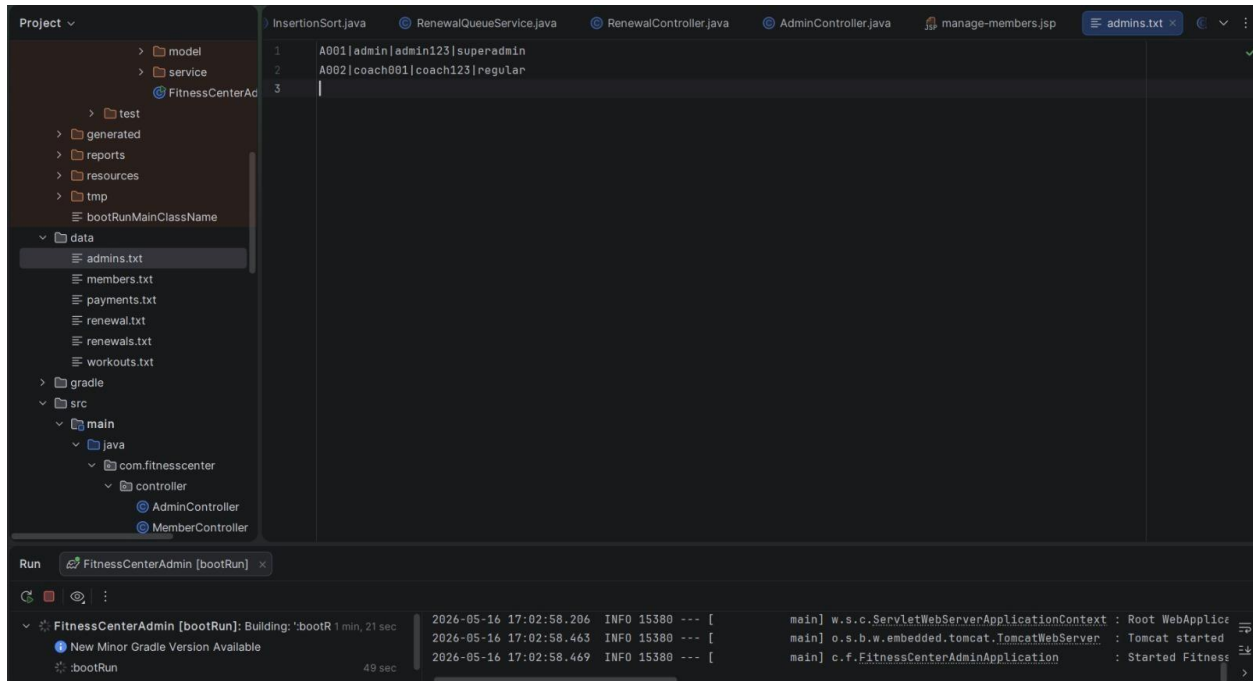
Run: FitnessCenterAdmin [bootRun] x

```

2026-05-16 17:02:58.296 INFO 15380 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplic
2026-05-16 17:02:58.463 INFO 15380 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started
2026-05-16 17:02:58.469 INFO 15380 --- [main] c.f.FitnessCenterAdminApplication : Started Fitness

```

(Figure 16- Members.txt)



(Figure 17 - Admin.txt)

Data is stored using file operations rather than a database, in accordance with the assignment requirements. The system uses structured text files (.txt) to store data, and Java file input/output streams such as `BufferedReader` and `BufferedWriter` are used to handle data storage and retrieval. Each module in the system is associated with a separate file, including **members.txt**, **admins.txt**, **workout.txt**, **dietplans.txt**, **payments.txt**, and **renewals.txt**. This modular file structure ensures organized data management across different system functionalities.

All file operations are managed through the service layer, where business logic is implemented. Whenever a user performs operations such as creating, updating, or deleting records, the corresponding service updates the relevant file. Similarly, when data needs to be displayed, the system reads and processes information from these files.

The system stores data in a structured format, where each record is written in a consistent pattern, making it easier to parse and reconstruct objects such as `Member`, `Payment`, `DietPlan`, and `RenewalRequest` during runtime.

This file-based storage approach ensures simple, efficient, and reliable data management while meeting the assignment requirements. It allows the system to maintain accurate records without the

need for a database management system, while still supporting dynamic updates and real-time data processing.

12. Testing and Validation

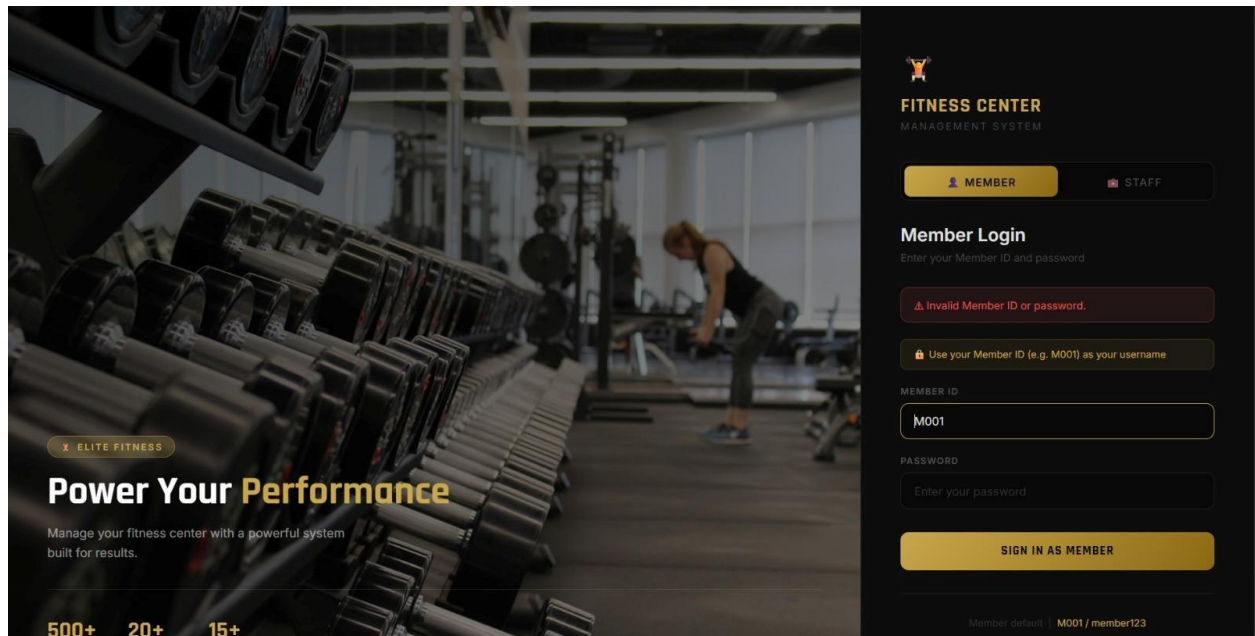
The system was tested extensively to verify the functionality and effectiveness of all its features. Particular emphasis was placed on the CRUD (Create, Read, Update, Delete) operations across all modules, including **Member Management, Workout and Diet Plan Management, Payment Management, Admin Management, and Renewal Management**. These operations were tested to ensure that data is correctly stored, retrieved, updated, and deleted using file handling mechanisms.

The system was also tested with various validation checks to improve reliability and usability. These validations included checks for empty input fields, correct data formats, and secure login credential verification to prevent errors and unauthorized access. Appropriate error messages were displayed for invalid inputs, while confirmation messages were provided for successful actions to clearly inform users about the status of their operations.

Additional testing was performed on the **renewal request processing system**, where the queue-based (FIFO) functionality was verified to ensure that requests are processed in the correct order. The **Insertion Sort algorithm** used for member prioritization was also tested to confirm that members are correctly arranged based on their membership expiry dates.

The **workout and diet plan modules** were tested to ensure that plans can be properly created, assigned, updated, and deleted for different members. Similarly, the **payment module** was tested to verify accurate recording, updating, and tracking of payment statuses.

As a result of these testing and validation processes, the system was confirmed to be **stable, accurate, reliable, and user-friendly**, ensuring smooth performance for both members and administrators.



(Figure 18- Login page Error)

13. GitHub Repository

The system was developed using GitHub as a version control platform to support collaboration among team members throughout the project development process. Each team member contributed by implementing their assigned modules based on a modular development approach and uploaded their source code to the repository through regular commits.

GitHub helped maintain proper project organization by enabling efficient code management, tracking updates, and reducing the risk of code conflicts during development. It allowed team members to work on different modules such as member management, payment processing, workout and diet planning, and renewal request handling simultaneously, while ensuring smooth integration of all components.

All project source files, including Java classes, Spring Boot controllers, service classes, HTML pages, CSS files, and configuration files, are stored in the repository. The modular structure of the project made it easier to manage and integrate individual contributions from each team member.

In addition, the commit history clearly records the changes made to different modules, demonstrating continuous development and contribution from each team member. This promotes transparency, accountability, and effective teamwork throughout the implementation of the fitness center management system.

14. Conclusion

In conclusion, the Fitness Center Management System was successfully developed as a web-based application using Java and Spring Boot, following Object-Oriented Programming (OOP) principles and the Model-View-Controller (MVC) architecture.

The system effectively integrates multiple modules such as member management, admin management, payment processing, workout and diet plan management, and renewal request handling, providing a complete and efficient solution for managing fitness center operations.

Through the development of this project, valuable knowledge and practical experience were gained in implementing CRUD operations, applying key OOP concepts such as encapsulation, inheritance, polymorphism, and abstraction, and utilizing file handling techniques for data storage and management. In addition, the system incorporates advanced features such as a queue-based (FIFO) renewal request system and the use of the Insertion Sort algorithm for member prioritization, demonstrating the application of data structures and algorithms in real-world scenarios.

The project also enhanced understanding of web application development using Spring Boot, HTML, CSS, and Bootstrap, while improving teamwork and collaboration skills through the use of GitHub for version control and module-based development.

Overall, the system demonstrates a well-structured, scalable, and user-friendly application that successfully meets the assignment requirements and reflects practical real-world software development practices.

Group members

IT25101711 - Madugalle E.W.M.W.S.A.K

IT25101719 - Jayawardena S.D.D.L

IT25101736 - Sooriyaarachchi A.G.S.T

IT25101744 - Wijewardhana A.A.G.S.U

IT25101763 - Wijewardana D.S
